

Scripts Documentation

Questa directory contiene tutti gli script di automazione per il deployment, manutenzione e gestione dell'infrastruttura di gioco.

Indice Scripts

Deployment Scripts

- `dev-reinstall.sh` - Setup completo ambiente sviluppo
- `prod-rebuild.sh` - Deploy produzione ottimizzato
- `update_frontend.sh` - Aggiornamento rapido frontend
- `update_sandbox.sh` - Aggiornamento ambiente test

Sistema di Pulizia Centralizzato

- `cleanup.sh` - Script di pulizia intelligente con modalità multiple
- `kibana-dashboard-manager.sh` - Gestione backup dashboard
- `check-persistent-config.sh` - Verifica configurazioni

 Documentazione Completa: [Sistema di Pulizia Centralizzato](#)

Utility Scripts

- `json_structure.py` - Analisi struttura project

Deployment Scripts

`dev-reinstall.sh`

Scopo: Setup completo ambiente di sviluppo con hot-reload e debugging

Operazioni eseguite:

1. Cleanup completo:

- Arresta tutti i container attivi
- Rimuove container, volumi anonimi e network
- Pulisce cache Docker

2. Reinstallazione dipendenze:

- Rimuove `node_modules` e lock files
- Reinstalla npm dependencies per frontend e sandbox
- Aggiorna package vulnerabilities

3. Build sviluppo:

- Build frontend in modalità development

- Configurazione sandbox con hot-reload
- Setup variabili ambiente per development

4. Avvio servizi:

- Start `docker-compose.dev.yml`
- Configurazione network per development
- Inizializzazione database con dati di test

Dipendenze Environment:

```
NODE_ENV=development      # Modalità sviluppo
VITE_PORT=5173             # Porta dev server
ELASTICSEARCH_PASSWORD    # Accesso monitoring
KIBANA_SYSTEM_PASSWORD    # Dashboard sviluppo
```

Utilizzo:

```
./scripts/dev-reinstall.sh
```

Tempo esecuzione: ~3-5 minuti **Output:** Tutti i servizi attivi con hot-reload abilitato

prod-rebuild.sh

Scopo: Deploy produzione con ottimizzazioni performance e sicurezza

Operazioni eseguite:

1. Production cleanup:

- Stop servizi esistenti
- Backup database prima della ricostruzione
- Cleanup risorse non necessarie

2. Build ottimizzato:

- Frontend build con tree-shaking e minification
- Asset optimization (immagini, fonts)
- Bundle splitting per performance

3. Deploy produzione:

- Start `docker-compose.prod.yml`
- Configurazione SSL e security headers
- Health check tutti i servizi

Dipendenze Environment:

```
NODE_ENV=production          # Modalità produzione
ELASTICSEARCH_PASSWORD      # Monitoring produzione
APM_SECRET_TOKEN            # Performance monitoring
PROXY_ADMIN_PORT            # Gestione proxy
```

Utilizzo:

```
./scripts/prod-rebuild.sh
```

Tempo esecuzione: ~4-7 minuti **Output:** Stack produzione ottimizzato e monitorato

update_frontend.sh

Scopo: Aggiornamento rapido solo frontend senza rebuild completo

Operazioni eseguite:**1. Frontend rebuild:**

- Rebuild solo container frontend
- Preserva stato database e cache
- Mantiene sessioni utente attive

2. Hot swap:

- Stop/start solo container frontend
- Zero downtime per altri servizi
- Sync immediato nuovi asset

Dipendenze Environment:

```
VITE_API_SERVER              # Backend endpoint
ELASTIC_APM_RUM_SERVER_URL   # Performance tracking
```

Utilizzo:

```
./scripts/update_frontend.sh
```

Tempo esecuzione: ~30-60 secondi **Output:** Frontend aggiornato con zero downtime backend

update_sandbox.sh

Scopo: Aggiornamento ambiente test isolato

Operazioni eseguite:

1. Sandbox refresh:

- Rebuild container sandbox
- Reset dati di test
- Configurazione networking isolato

Utilizzo:

```
./scripts/update_sandbox.sh
```

Sistema di Pulizia e Manutenzione

Il sistema di pulizia è stato centralizzato nello script `cleanup.sh` che offre modalità multiple per diversi scenari.

Vedere [Sistema di Pulizia Centralizzato](#) per la documentazione completa.

- Preparazione rilasci clean

`cleanup.sh`

Scopo: Sistema di pulizia centralizzato e intelligente con modalità multiple

Modalità Disponibili:

`--dev` (Modalità Sviluppo - Default)

Pulizia completa per ambiente di sviluppo:

- Backup automatico configurazioni critiche
- Rimozione container e volumi Docker
- Pulizia artifacts di build (node_modules, dist, cache)
- Preserva configurazioni essenziali

`--production` (Modalità Produzione)

Pulizia selettiva per deployment produzione:

- Backup automatico prima della pulizia
- Rimozione solo dati di sviluppo
- Preserva TUTTE le configurazioni
- Verifica integrità post-pulizia

`--soft` (Modalità Leggera)

Pulizia conservativa:

- Solo cache e build artifacts
- Preserva dati e configurazioni
- Ideale per pulizie quotidiane

--full (Modalità Completa - ATTENZIONE!)

Pulizia totale con conferma esplicita:

- Cancella TUTTO comprese configurazioni
- Richiede digitare 'DELETE_ALL'
- Backup automatico prima della distruzione

Uso:

```
./scripts/cleanup.sh --dev           # Pulizia sviluppo (default)
./scripts/cleanup.sh --production    # Pulizia per produzione
./scripts/cleanup.sh --soft          # Solo cache
./scripts/cleanup.sh --full          # TUTTO (attenzione!)
./scripts/cleanup.sh --help         # Mostra help
```

Caratteristiche:

-  **Backup automatico** configurazioni critiche
-  **Protezione intelligente** certificati SSL, NGINX, Kibana
-  **Output colorato** e informativo
-  **Verifica integrità** post-operazione
-  **Modalità multiple** per ogni scenario

kibana-dashboard-manager.sh

Scopo: Gestione completa backup e ripristino dashboard Kibana personalizzate

Operazioni:

- **Export:** Esporta tutte le dashboard correnti
- **Import:** Importa dashboard da file backup
- **List:** Lista backup disponibili
- **Protect:** Configura protezione negli script di pulizia

Uso:

```
./scripts/kibana-dashboard-manager.sh export
./scripts/kibana-dashboard-manager.sh import dashboard-backup.ndjson
./scripts/kibana-dashboard-manager.sh import-latest
./scripts/kibana-dashboard-manager.sh list
```

check-persistent-config.sh

Scopo: Verifica integrità configurazioni persistenti dopo deployment

Operazioni eseguite:

1. Environment verification:

- Controlla file `.env` e `docker/env/.env`
- Verifica configurazioni Elasticsearch/Kibana
- Valida token APM e credenziali

2. Services health check:

- Verifica connettività database
- Check status Elasticsearch cluster
- Valida configurazioni proxy

3. Persistent data verification:

- Controlla volumi Docker
- Verifica backup configurations
- Valida integrità database

Controlli eseguiti (51 totali):

-  **Environment Files:** `.env`, `docker/env/.env`
-  **NGINX Proxy Manager:** Configurazioni SSL e domini
-  **Elasticsearch:** Data integrity e cluster health
-  **Kibana:** Configurazioni dashboard e index patterns
-  **APM Server:** Token validation e connectivity
-  **Elastic Agent:** System monitoring configuration
-  **Fluent Bit:** Log aggregation e parsing rules
-  **MongoDB:** Data persistence e backup status
-  **Redis:** ACL configuration e performance
-  **Frontend Configuration:** APM integration e environment
-  **Docker Compose:** Service definitions e networking

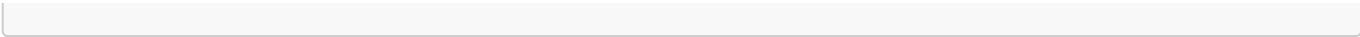
Output esempio:

```
📊 Check Results:
Total Checks: 51
✅ Passed: 47
❌ Failed: 0
⚠ Warnings: 4

🎉 All persistent configurations are intact!
```

Utilizzo:

```
./scripts/check-persistent-config.sh
```



Dipendenze Environment:

```
ELASTICSEARCH_USERNAME      # Accesso cluster
ELASTICSEARCH_PASSWORD      # Validazione connessione
APM_SECRET_TOKEN            # Verifica APM integration
KIBANA_SYSTEM_PASSWORD      # Dashboard access
```



 Utility Scripts

json_structure.py

Scopo: Analisi struttura progetto e dipendenze

Operazioni eseguite:

- 1. Project analysis:
 - Scansione directory structure
 - Analisi dipendenze package.json
 - Report configurazioni Docker

Utilizzo:

```
python scripts/json_structure.py
```



 Script Development Guidelines

Convenzioni

- Tutti gli script bash includono error handling
- Output colorato per feedback utente
- Logging operazioni critiche
- Rollback automatico in caso di failure

Environment Dependencies

Ogni script dipende da variabili specifiche in .env:

```
# Variabili comuni a tutti gli script
ELASTICSEARCH_PASSWORD      # Accesso monitoring
KIBANA_SYSTEM_PASSWORD      # Dashboard access
APM_SECRET_TOKEN            # Performance tracking
PROXY_ADMIN_PORT            # Proxy management
```

```
# NPM Registry authentication
NPM_EMAIL           # npm login credentials
NPM_PASSWORD        # npm access token/password

# Development specific
NODE_ENV=development
VITE_PORT=5173

# Production specific
NODE_ENV=production
SSL_ENABLED=true
```

Error Handling

Tutti gli script implementano:

- Controllo prerequisiti (Docker, Node.js)
- Validation variabili ambiente
- Cleanup automatico in caso di errore
- Log dettagliati per debugging

Performance Optimization

- Build parallelizzati dove possibile
- Cache intelligente per speed up
- Health check non-blocking
- Cleanup risorse automatico



Script Monitoring

Ogni script può essere monitorato tramite:

1. **Log output:** Tutti gli script generano log strutturati
2. **Exit codes:** Codici di uscita standardizzati per automation
3. **Metrics:** Performance timing per ogni operazione
4. **Health checks:** Validation automatica post-execution

Per monitoring automatico:

```
# Esecuzione con logging
./scripts/dev-reinstall.sh 2>&1 | tee deployment.log

# Check exit code
echo $? # 0 = success, >0 = error
```

dev-reinstall.sh

Full development environment reinstall from scratch

- Cleans JavaScript artifacts (dist, node_modules, lockfiles)
- Reinstalls all JS dependencies
- Updates Go modules in isolated container
- Builds frontend bundles
- Removes Docker containers and images
- **Preserves all persistent configurations and data**
- Rebuilds and starts the entire stack

Usage: `./scripts/dev-reinstall.sh`

What's Preserved:

- Kibana dashboards and Fleet configurations
- Elasticsearch indices and APM data
- MongoDB application data
- NGINX Proxy Manager settings and SSL certificates
- Redis ACL configurations

`check-persistent-config.sh`

Verifies persistent data integrity

- Checks all critical configuration files exist
- Reports on persistent data directory contents
- Validates environment variable synchronization
- Verifies Docker Compose configuration validity

Usage: `./scripts/check-persistent-config.sh`

`dev-rebuild.sh`

Quick rebuild without cleaning dependencies

- Rebuilds Docker images only
- Faster than full reinstall
- Use when only Docker configuration changes

Script di Pulizia Centralizzato

Per tutte le operazioni di pulizia utilizzare il [Sistema di Pulizia Centralizzato](#):

- `./scripts/cleanup.sh --dev` - Pulizia completa sviluppo
- `./scripts/cleanup.sh --production` - Pulizia selettiva produzione
- `./scripts/cleanup.sh --soft` - Solo cache e artifacts
- `./scripts/kibana-dashboard-manager.sh` - Gestione dashboard

`update_frontend.sh / update_sandbox.sh`

Individual service updates

- Updates specific frontend services

- Rebuilds only affected containers

Important Notes

⚠ **Data Persistence:** The `dev-reinstall.sh` script is designed to preserve all manual configurations made through Kibana, Fleet, NGINX Proxy Manager, and other UIs. These configurations are automatically saved to local directories in `docker/volumes/`.

🔑 **Configuration Sync:** Always ensure both `.env` files (root and `docker/env/.env`) remain synchronized.

🚀 **Quick Start:** For new developers, run `./scripts/check-persistent-config.sh` first to verify the environment, then `./scripts/dev-reinstall.sh` for a complete setup.

📖 Development Guidelines

⚠ Per le complete best practices di sviluppo, architettura e deployment consultare: [📖 GAME_ARCHITECTURE.md](#)

Il documento centralizza:

- 🏗 **Architettura & Design Patterns**
- 💻 **Frontend/Backend Best Practices**
- 🛡 **Security & Schema Validation**
- 📊 **Monitoring & Analytics Setup**
- ⚡ **Performance Optimization**
- 🧪 **Testing Strategies**
- 🚀 **Deployment & CI/CD**

Quick Reference - Essential Commands

Development Environment:

```
./scripts/dev-reinstall.sh           # Full environment setup
./scripts/check-persistent-config.sh # Verify configuration
./scripts/update_frontend.sh         # Quick frontend update
./scripts/cleanup.sh --soft          # Light cleanup (cache only)
```

Production Deployment:

```
./scripts/kibana-dashboard-manager.sh export # Backup dashboards
./scripts/cleanup.sh --production            # Clean development data
./scripts/check-persistent-config.sh         # Verify configurations
./scripts/prod-rebuild.sh                   # Deploy production
```

Maintenance:

```
./scripts/cleanup.sh --dev          # Full development cleanup
./scripts/kibana-dashboard-manager.sh list      # List dashboard backups
```

Performance Monitoring:

```
# Check game performance
curl http://localhost/health
curl http://kibana.localhost/api/status

# Monitor real-time logs
docker logs server --tail 50 -f      # API server
docker logs frontend --tail 50 -f    # Frontend build
docker logs fluent-bit --tail 20 -f  # Log aggregation
```

Production Deployment:

```
./scripts/prod-rebuild.sh          # Full production build
# Uses docker-compose.prod.yml (excludes sandbox)
```

Schema Validation System

Il sistema di validazione automatica ([comms/server/public/](#)) garantisce:

- **Route Whitelisting:** Solo endpoint con schema JSON sono accessibili
- **Request Validation:** Validazione automatica body contro schema
- **Security by Default:** Rotte non schemate vengono bloccate
- **GDPR Compliance:** Validazione consent obbligatoria

Monitoring & Analytics Integration

Frontend Analytics (esempio eventi essenziali):

```
// Level progression tracking
analytics.logEvent('level_completed', {
  level: 3, completion_time: 45.2, score: 1250, deaths: 2
});

// Performance monitoring
analytics.logEvent('performance_issue', {
  issue_type: 'low_fps', fps: 28, level: 5
});

// Achievement tracking
analytics.logEvent('achievement_unlocked', {
  achievement_id: 'speed_runner', level: 3
});
```

Backend Structured Logging:

```
// APM tracking obbligatorio per nuovi endpoint
func (h *GameHandler) SaveProgress(ctx context.Context, req
*SaveProgressRequest) error {
    span, ctx := apm.StartSpan(ctx, "game.save_progress",
"business_logic")
    defer span.End()

    // Structured logging con context
    logger.WithFields(logrus.Fields{
        "player_id": req.PlayerID,
        "level": req.Level,
        "operation": "save_progress",
    }).Info("Processing progress save")

    return h.gameService.SaveProgress(ctx, req)
}
```

Critical Development Requirements

1. Testing Strategy:

- ☒ Unit tests per game logic
- ☒ Integration tests per API endpoints
- ☒ Performance validation (FPS ≥50, API <100ms)
- ☒ Schema validation testing

2. Monitoring Requirements:

- ☒ APM tracing per tutti i nuovi endpoint
- ☒ Analytics events per interazioni utente significative
- ☒ Performance metrics tracking
- ☒ Error tracking e logging strutturato

3. Deployment Checklist:

- ☒ Performance benchmarks rispettati
- ☒ Test su frontend reale (non solo sandbox)
- ☒ APM tracking configurato e verificato
- ☒ Analytics events validati in Kibana
- ☒ Schema validation aggiornata
- ☒ Health checks funzionanti

 Per dettagli completi su implementazione, esempi di codice e troubleshooting consultare [GAME_ARCHITECTURE.md](#)